Nitsche speedup item 8a: dense stiffness blocks

How the two stiffness products of each step now come from dense element and face blocks, how the change was checked against `d30c76e`, and what it gained on TPV205, TPV104 and TPV102 at p = 1 and p = 2 · revision 1.1, 18 Oct 2026

Revision 1.1, 18 Oct 2026. Adds the Vista results (§6): build job 1079599 and speed job 1079640, both at `7cab085`, the speed job on 2304 ranks. The code is now committed on branch `claude/nitsche-profiling-w336yq` as `241cc80` (the dense blocks) and `7cab085` (the Vista job), pushed after Chunhui Zhao's approval; file:line citations refer to `241cc80` and link to GitHub. §1, §5.1, §7 and §8 are updated to match; the cloud results of §4 and §5 are unchanged. Revision 1 (same day) was the review version. Timings in §4 and §5 are from the cloud VM (4 cores, Xeon at 2.1 GHz, one rank per core); §6 is Vista. Item numbers and phases are those of the design document `lts-kernels-design-rev1.pdf` (cited as "design §n"), and items 1 to 6 are those of `nitsche-speedups-impl-rev1.1.pdf`.

1. Summary

Item 8a is in and on by default. On the cloud VM with 4 ranks, a time step is **1.64 to 2.24 times faster** than at `d30c76e` on all seven cases, and a whole run is 1.41 to 1.69 times faster (§5, Figure 2). On 1 rank the step is 1.78 times faster. The design predicted 1.8 to 2.2 times per step (design §4). Six of the seven cases fall in that range, p = 2 at its top; TPV102 1000 m at p = 1 is below it (§5.2).

On Vista (§6), with 2304 ranks on TPV205 200 m at p = 2, a step takes **9.36 ms**, against 16.91 ms with the previous defaults (**1.81 times faster**) and 27.28 ms with every speed option off (2.91 times). That is at the bottom of the design's range and below the cloud's 2.20 times for TPV205 at p = 2. The 1 s run takes 197 s instead of 257 s; setup (about 125 s, as with the previous defaults) did not get faster and is now most of it. From the step times, a full 12 s run of this case would take about 17 minutes instead of about 45 (inferred, §6.3). The run with every option off is bitwise equal to gold 1075916 at all 16 stations, and the dense run differs from it by at most $2.7 \cdot 10^{-11}$.

The answers do not change. With `--dense-blocks 0` every case matches `d30c76e` bit for bit, at every station and in every fault field. With the dense blocks on, the products equal the assembled ones to at most $1.1 \cdot 10^{-15}$ of their largest entry, the stations differ from `d30c76e` by at most $6.2 \cdot 10^{-11}$ of each quantity's largest value, and the fault fields by at most $5.6 \cdot 10^{-12}$ (§4). The six gate test sets pass with the same counts as before.

The gain comes from reading less memory. Stored as dense blocks, K and K_η together take 5.8 kB per tet at p = 1 and 35 kB at p = 2, against 17 kB and 104 kB for the two CSR matrices (2.9 times less), and one read of each block gives both products. The products alone are 2.4 to 2.8 times faster than the two hypre products and 1.9 to 2.4 times faster than the fused sweep of item 4 (§5). Memory while stepping falls 1.6 to 4.2 times, and the setup peak falls by up to 21%, because item 4's union copy is no longer built.

What this needs from you: nothing for the code, which is committed and pushed with the dense blocks on by default. This revision, the speed job's output and the README rows for both Vista jobs are proposed as one documentation commit (§8).

2. Where the change sits

The change replaces one call in the step and adds one setup call (Figure 1). Everything else in the step, the fault work included, is unchanged.

File (under `miniapps/seas/`)	Change
`domain/dense_block_stiffness.hpp` (new, 540 lines)	Class `DenseBlockStiffness`: `Init` reads the assembled hypre matrices into blocks; `Mult` applies them
`domain/nitsche_wave_operator.hpp` (+63 −2)	`SetDenseBlocks`, `DenseBlocksActive`, `DenseBlockBytes`; `UseDense_` builds the blocks on the first acceleration; `Acceleration_` calls them before item 4's sweep
`drivers/nitsche_dyn_driver.cpp` (+10 −2)	Flag `--dense-blocks 0|1` (default 1); the log line says whether the blocks are on and their size
`tests/unit/bench_nitsche_kernels.cpp` (+89)	Section [D]: correctness checks and timings against the hypre products and item 4
`Makefile` (+1)	The new header in `DOMAIN_HEADERS`
`jobs/vista_profiling/` (commit `7cab085`)	New job `speed_dense_tpv205_200m_p2.sbatch`; README rows for it (§6)

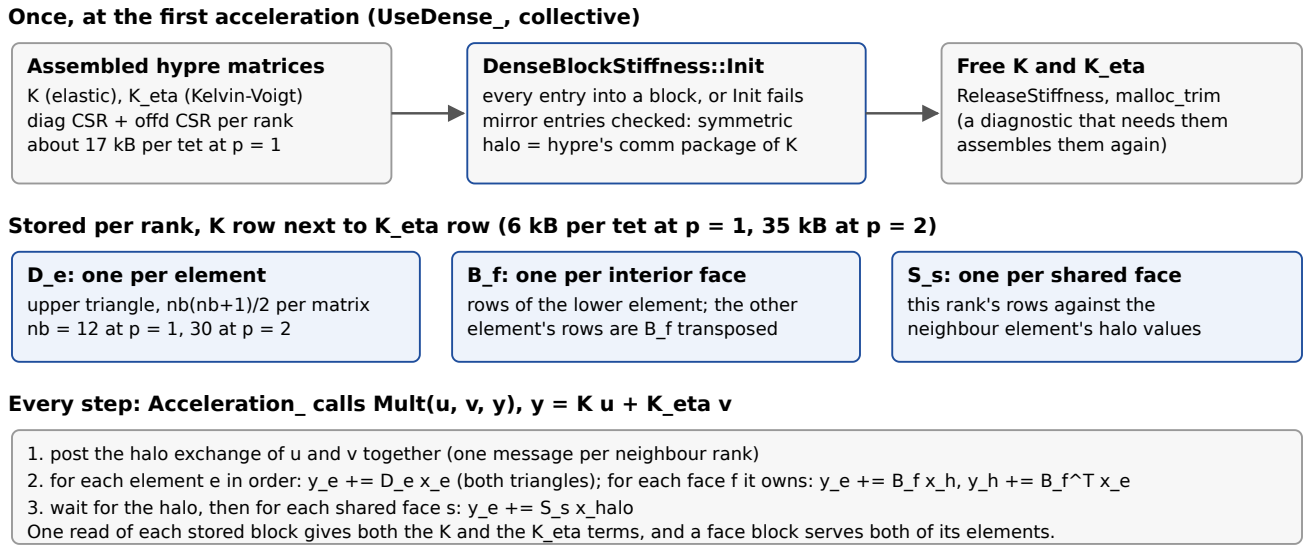


Figure 1. What item 8a builds once and what it does every step. x_e holds the values of u and v at element e; x_h those at the other element of face f.

3. How it works

3.1 Workflow

1. The driver calls `op.SetDenseBlocks(dense_blocks != 0)` next to `SetFusedStiffness` (`nitsche_dyn_driver.cpp:1504`).
2. The first call of `Acceleration_` (the initial acceleration) calls `UseDense_` (`nitsche_wave_operator.hpp:935-962`). On every rank it runs `DenseBlockStiffness::Init` on K, and on K_η when the deck has Kelvin-Voigt damping, then reduces the success flag with

`MPI_MIN`. On success it frees both hypre matrices, as item 4 does. On failure on any rank, every rank keeps the old path: item 4's fused sweep with damping, the hypre product without.

3. Each step, `Acceleration_` (lines 843-855) computes $Ku + K_\eta v$ with `dense_.Mult(x, *v, Kx_)`, or Ku with `dense_.Mult(x, Kx_)` when there is no damping, and skips the separate $K_\eta v$ product. The rest of the residual, $F - Ku - K_\eta v + Gd + G_\eta V - Cv$, is unchanged.

3.2 `DenseBlockStiffness::Init` (dense_block_stiffness.hpp:59-293)

What it does. It reads the assembled matrices into dense blocks and checks that nothing was left out. A DG row of element e couples only to e itself and to the elements that share a face with it, so each nonzero belongs to an element block or a face block. Init places every entry; if one has nowhere to go, it returns false and the caller keeps the CSR products.

Inputs: the DG space (one block size for all elements, nodes ordering, true DOFs = local DOFs), K, and K_η or null. **Output:** true when every entry was placed and both matrices are symmetric to 10^{-12} of their largest entry; the blocks; a copy of hypre's halo pattern.

```
Init(fes, K, E):
  exchange face-neighbour data; make hypre's comm packages
                                        # collective, before any rank can return
  each element e: its nb DOFs           # nb = 12 at p = 1, 30 at p = 2
  interior faces: pairs (e1 < e2)       # a face is owned by its lower element
  shared faces: pairs (local e, neighbour element);
                each halo column -> (neighbour, position)
  for K and E:
    diag CSR, pass 1: (i, j) inside element e  -> D_e[a, b] if a <= b
                      (i, j) between e < g     -> B_f[a, b]
                      anything else            -> bad
    diag CSR, pass 2: each skipped mirror entry vs the stored one -> asymmetry
    offd CSR:         (i, halo column)         -> S_s[a, position], else bad
  return bad == 0 and asymmetry <= 1e-12 max|entry|
```

Why symmetric storage is valid. The volume part of K is symmetric, and `DGElasticityIPCombinedIntegrator` sets each face matrix to $(A + A^\top)/2$ (`integrator/dg_elasticity_ip_combined_integrator.hpp:183-195`). The assembled matrices are therefore symmetric up to the order of their sums. The bench measures the largest $|A_{ij} - A_{ji}|$ at 4.7 to 6.9·10^{-17} of the largest entry (§4). Init refuses a matrix that is not symmetric to 10^{-12}, so a future unsymmetric variant would fall back to the CSR products rather than give wrong answers.

Layout. The rows are stored one after another, a row of K followed by the same row of K_η. An element keeps its upper triangle row by row ($n_b(n_b+1)/2$ values per matrix). A face keeps the full $n_b \times n_b$ rows of its lower element. A shared face keeps this rank's rows against the neighbour's DOFs. A halo column with no entry points at a zero slot.

3.3 `Mult` (dense_block_stiffness.hpp:316-326, 416-522)

What it does. It computes $y = Ku$ or $y = Ku + K_\eta v$ in one sweep over the elements. **Inputs:** u (and v) on this rank's DOFs. **Output:** y on this rank's DOFs. It is collective: it exchanges the halo values of u and v in one message per neighbour rank, as item 4 does.

```
Mult(u, v, y):
  post Irecv / Isend of the halo values of u and v   # overlaps the local work
  y = 0
  for e = 0 .. ne-1:              # y_e already holds the B^T terms of e's
    x_e = (u, v) at e             # lower-numbered neighbours
    acc = y_e + D_e x_e           # upper triangle: D[a,b] used for row a and row b
    for each face f owned by e, other element h:
      x_h = (u, v) at h
      acc += B_f x_h ;  y_h += B_f^T x_e      # one read of B_f gives both
    y_e = acc
  wait for the halo
  for each shared face s of element e:  y_e += S_s x_halo
```

The block sizes of p = 1 and p = 2 (12 and 30) get fixed-size loops; other orders run a generic loop (up to p = 5). The row loops use two-wide vectors with two independent partial sums (`RowDotAxpy_`, `RowDot_`, lines 346-404): one chain of multiply-adds was bound by its latency. On a microbenchmark (TPV205 5000 m, p = 1, 1 rank) this took $Ku + K_\eta v$ from 14.4 to 12.2 ms. Two-wide vectors match the 128-bit NEON registers of Vista's Grace cores.

Why the bits change. The entries are the assembled ones, but the sums run in a different order, and the lower triangle and the transposed face rows use the stored mirror entries, which differ from the assembled ones by at most $6.9 \cdot 10^{-17}$ of the largest entry. Both are round-off. A rupture run amplifies them, here by four to five orders of magnitude (§4.3).

3.4 Operator and driver

- `SetDenseBlocks(bool)` (line 726): on by default; call before the first step. `DenseBlocksActive()` is true once the blocks are in use; `DenseBlockBytes()` is their size summed over the ranks.
- `UseDense_` rebuilds the blocks if Kelvin-Voigt is switched on after the first read, so the two-matrix and one-matrix forms cannot be mixed up (line 937).
- The order is dense blocks first, then item 4's sweep (`--fused-spmv`, used only when `--dense-blocks 0`), then the hypre products.
- The driver's timing line now ends with `dense blocks on (N MB over all ranks)` or `dense blocks off` (line 1654).

3.5 Bench section [D]

`seas_bench_nitsche_kernels` gains section [D] (bench_nitsche_kernels.cpp:693-767). On the production K and K_η of the TPV205 deck it checks, on any number of ranks, that Init accepts both matrices, that Ku, $Ku + K_\eta v$ and $Ku + K_\eta u$ equal the hypre products to 10^{-13}, and it times the blocks in alternation against the two hypre products, against item 4's sweep, and Ku alone. The Vista build job runs this bench on 16 ranks, which is the design's np = 16 check (phase P4); build 1079599 passed it (§6.5).

4. Checks: the results are unchanged

4.1 The products (bench [D])

Run	Checks	Ku	$Ku + K_\eta v$	$Ku + K_\eta u$	Asymmetry K, K_η
1 rank, TPV205 2000 m, p = 1	18 of 18	$4.4 \cdot 10^{-16}$	$5.0 \cdot 10^{-16}$	$4.9 \cdot 10^{-16}$	$6.9 \cdot 10^{-17}$, $4.7 \cdot 10^{-17}$
4 ranks, TPV205 2000 m, p = 1	14 of 14	$5.1 \cdot 10^{-16}$	$6.3 \cdot 10^{-16}$	$5.0 \cdot 10^{-16}$	$6.9 \cdot 10^{-17}$, $4.7 \cdot 10^{-17}$
4 ranks, TPV205 5000 m, p = 2	14 of 14	$1.0 \cdot 10^{-15}$	$1.1 \cdot 10^{-15}$	$8.9 \cdot 10^{-16}$	$5.0 \cdot 10^{-17}$, $4.9 \cdot 10^{-17}$
1 rank, TPV205 5000 m, p = 2	18 of 18	$1.1 \cdot 10^{-15}$	$1.1 \cdot 10^{-15}$	$1.1 \cdot 10^{-15}$	$5.0 \cdot 10^{-17}$, $4.9 \cdot 10^{-17}$

The differences are the largest $|y_{new} - y_{hypre}|$ over the largest $|y_{hypre}|$, the design's acceptance test ($\leq 10^{-13}$, design §4). On 4 ranks the 2000 m mesh has 4344 shared-face blocks and the 5000 m mesh 1294, so the halo path is covered. The 1-rank runs pass every other bench section as well (18 checks against 14 on 4 ranks, because some sections need every fault face on one rank). The p = 2 bench on the 3000 m mesh did not fit: 4 ranks at 3.6 GB each exceeded the VM's memory limit and the kernel killed the bench (cgroup out of memory); the 5000 m mesh was used instead.

4.2 Gate tests

The six gate test sets of the Vista build job pass in the cloud with the new driver library: `test-nitsche-gates` 74, `test-nitsche-pf` 22, `test-nitsche-hook-spy` 49, `test-nitsche-lsw` 13, `test-nitsche-srw` 9, `test-nitsche-lsw-inline` 42 passed, 0 failed: the same counts as Vista build 1078824.

4.3 Full runs against `d30c76e`

Each case ran three times on the same VM, one after the other: the driver built from `d30c76e`, the new driver with its defaults (dense blocks on), and the new driver with `--dense-blocks 0`. The decks are the cloud copies of the repository decks used for items 1 to 6 (TPV205 and TPV104 with Kelvin-Voigt 0.1 and Prakash-Clifton, TPV102 likewise); the undamped TPV205 case sets `kv_eta = 0` so that the K-only path runs. Station differences are the largest over all stations and columns of $|a - b|$ over the column's largest $|a|$; fault-field differences are the same over every field of every frame of the `.vtkhdf` file.

Case	`--dense-blocks 0` vs d30c76e	Dense vs d30c76e: stations	Dense vs d30c76e: fault fields
TPV205, 2000 m, p = 1	bitwise equal (stations and all fault fields)	$7.8 \cdot 10^{-12}$	$9.4 \cdot 10^{-14}$
TPV205, 2000 m, p = 1, no damping	bitwise equal (stations and all fault fields)	$6.2 \cdot 10^{-11}$	$5.6 \cdot 10^{-12}$
TPV205, 3000 m, p = 2	bitwise equal (stations and all fault fields)	$1.9 \cdot 10^{-11}$	$3.3 \cdot 10^{-14}$
TPV104, 3000 m, p = 1	bitwise equal (stations and all fault fields)	0	$5.7 \cdot 10^{-15}$
TPV104, 3000 m, p = 2	bitwise equal (stations and all fault fields)	0	$5.0 \cdot 10^{-16}$
TPV102, 1000 m, p = 1	bitwise equal (stations and all fault fields)	$1.3 \cdot 10^{-12}$	$9.0 \cdot 10^{-16}$
TPV102, 3000 m, p = 2	bitwise equal (stations and all fault fields)	$1.7 \cdot 10^{-13}$	$3.2 \cdot 10^{-16}$
TPV205, 2000 m, p = 1, 1 rank	not run	$9.1 \cdot 10^{-12}$	$1.5 \cdot 10^{-13}$

With the switch off the new driver is the old code: every station file and every fault field is bitwise equal to `d30c76e`. With it on, the differences are round-off grown by the run, of the size items 3 and 4 gave ($6.6 \cdot 10^{-11}$ on Vista, impl rev1.1 §7). TPV104 stations agree to the digits the files store. A new gold is not needed.

5. Measured gains

5.1 The products

Run	Two hypre products	Item 4 sweep	Dense blocks	vs hypre	vs item 4	Ku alone: hypre → dense
1 rank, 2000 m, p = 1	213.9 ms	170.9 ms	87.8 ms	2.44×	1.99×	102.6 → 45.8 ms (2.24×)
4 ranks, 2000 m, p = 1	46.6 ms	39.5 ms	19.5 ms	2.40×	1.92×	22.6 → 10.4 ms (2.18×)
1 rank, 5000 m, p = 2	173.8 ms	145.2 ms	62.9 ms	2.76×	2.31×	84.8 → 32.1 ms (2.64×)
4 ranks, 5000 m, p = 2	42.7 ms	37.5 ms	15.0 ms	2.84×	2.40×	21.9 → 8.1 ms (2.72×)

Fastest of 60 (p = 1) or 30 (p = 2) alternating calls; the medians give ratios within 0.17 of these. The bytes per tet are 5.81 kB against 17.06 kB at p = 1 and 34.96 against 104.0 kB at p = 2. On this VM the blocks are read at about 4 GB/s (p = 1) and 6 GB/s (p = 2) per rank against 12 GB/s for a triad, so here the kernel is limited by its arithmetic and gathers rather than by memory (inferred from bytes and time). On a full Vista node the 144 ranks share the memory, so the ratio there could differ in either direction; §6 gives the Vista numbers.

5.2 Whole runs

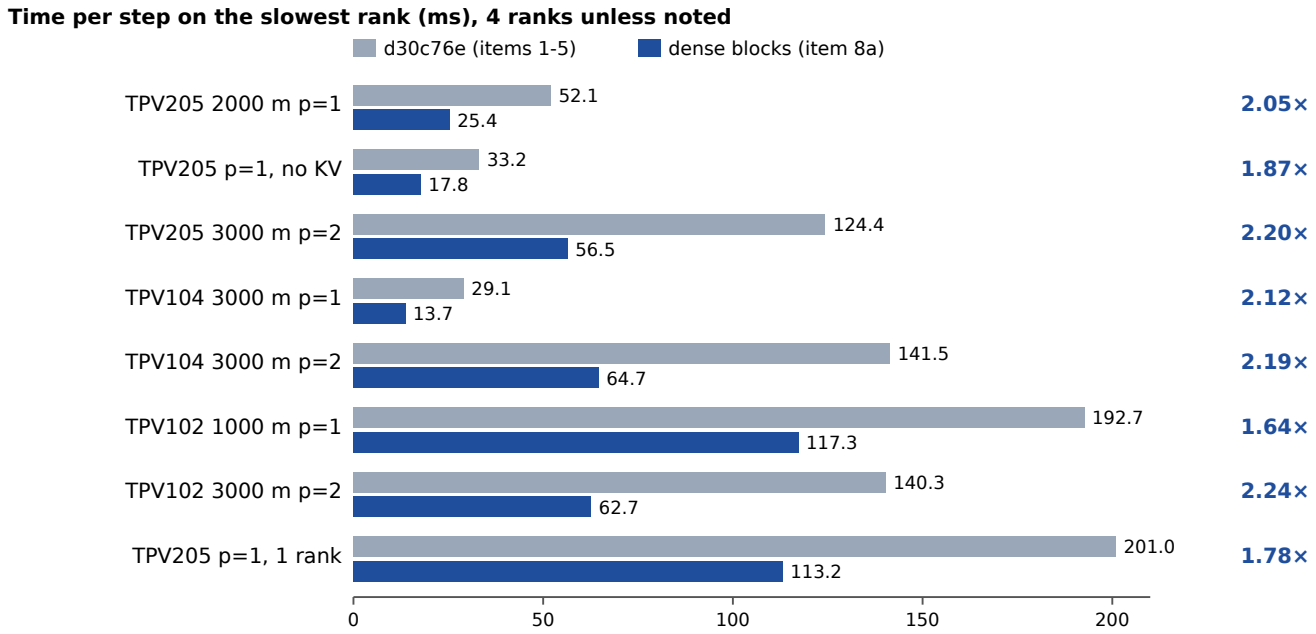


Figure 2. Time inside `ode.Step` per step after step 1, on the slowest rank. The bold number is the speedup.

Case	Steps	ms per step, d30c76e	ms per step, dense	Step speedup	Wall, d30c76e → dense (s)	Step 1 with setup (s)
TPV205, 2000 m, p = 1	369	52.1	25.4	**2.05×**	25.9 → 15.3 (1.69×)	5.7 → 4.9
TPV205, 2000 m, p = 1, no damping	369	33.2	17.8	**1.87×**	15.4 → 10.0 (1.54×)	2.2 → 2.6
TPV205, 3000 m, p = 2	211	124.4	56.5	**2.20×**	47.1 → 29.8 (1.58×)	20.3 → 17.3
TPV104, 3000 m, p = 1	217	29.1	13.7	**2.12×**	9.2 → 6.0 (1.52×)	2.4 → 2.6
TPV104, 3000 m, p = 2	232	141.5	64.7	**2.19×**	55.8 → 35.2 (1.59×)	22.5 → 19.5
TPV102, 1000 m, p = 1	336	192.7	117.3	**1.64×**	95.6 → 67.9 (1.41×)	26.0 → 24.9
TPV102, 3000 m, p = 2	232	140.3	62.7	**2.24×**	55.4 → 34.1 (1.63×)	22.1 → 18.8
TPV205, 2000 m, p = 1, 1 rank	369	201.0	113.2	**1.78×**	93.9 → 62.7 (1.50×)	18.2 → 19.2

Step 1 includes the operator assembly and the block build. It does not get slower: the blocks are built from matrices that were assembled anyway, and item 4's union copy is no longer made. The TPV102 1000 m case gains least (1.64 times). Its fault work is only 6% of its step (7.2 ms of 117 ms), so the larger remainder is other work outside the two products; which

part, I have not profiled. The `--dense-blocks 0` runs took 26.9 to 200.5 ms per step, within 9% of `d30c76e` in both directions, which is the VM's run-to-run spread.

5.3 Memory

Case	Stepping memory, all ranks (GiB)	Setup peak (GiB)	Dense blocks (MB)
TPV205, 2000 m, p = 1	1.18 → 0.68 (1.7× less)	2.60 → 2.45 (-5.6%)	369
TPV205, 2000 m, p = 1, no damping	1.94 → 0.47 (4.2× less)	1.94 → 1.93 (-0.5%)	187
TPV205, 3000 m, p = 2	2.84 → 1.38 (2.1× less)	7.77 → 7.29 (-6.2%)	1078
TPV104, 3000 m, p = 1	0.67 → 0.41 (1.6× less)	1.40 → 1.33 (-5.2%)	192
TPV104, 3000 m, p = 2	3.09 → 1.50 (2.1× less)	9.51 → 7.94 (-16.6%)	1176
TPV102, 1000 m, p = 1	3.76 → 2.04 (1.8× less)	8.68 → 8.15 (-6.1%)	1275
TPV102, 3000 m, p = 2	3.08 → 1.50 (2.1× less)	10.03 → 7.94 (-20.9%)	1176
TPV205, 2000 m, p = 1, 1 rank	1.06 → 0.56 (1.9× less)	3.09 → 2.44 (-21.2%)	364

Stepping memory is the median of the summed resident memory of the ranks over the last 40% of the run; the setup peak is the largest 0.2 s sample. Without damping, item 4 never ran, so `d30c76e` kept the hypre K; the blocks free it, hence the 4.2 times there.

6. Vista: 2304 ranks

6.1 The two jobs

Both jobs ran in the clone `$WORK/seas-mfem-profiling` at `7cab085` (clean tree, checked by the job guard and again before the speed job was submitted), account EAR26024, partition `gg`, modules gcc/14.2.0, cuda/12.6, openmpi/5.0.5, hypre/2.31.0, metis/5.1.0.3, phdf5/1.14.6.

- **Build 1079599** (`build_profiling_vista.sbatch`, 1 node, 3 min 22 s): the driver, the gate tests and the bench built at `7cab085` with no tracked changes; the six gate test sets passed (74, 22, 49, 13, 9, 42 passed, 0 failed); the 200 m mesh matched the sha256 of gold 1075916; the 16-rank bench passed 14 of 14 checks (§6.5).
- **Speed 1079640** (`speed_dense_tpv205_200m_p2.sbatch`, 16 nodes × 144 ranks, 13 min 35 s, exit 0): TPV205, 200 m gold mesh (sha256 `30c63b92…`), p = 2, 0 to 1.0 s, deck `tpv205/configs/tpv205_nitsche.toml`, 7394 steps. One allocation ran three times:

Run	Flags	What it is
`off`	`--fast-fault 0 --fused-spmv 0 --fast-fault-output 0 --dense-blocks 0`	The code as at gold 1075916
`item4`	`--dense-blocks 0`	The defaults before item 8a (items 1, 3, 4, 5)
`dense`	none	The defaults at `7cab085`: items 1, 3, 5 and the dense blocks

6.2 The results are unchanged

Comparison (16 stations, slip, V, τ, σ_n)	Largest difference	Where
`off` against gold 1075916 (first 1.0 s)	**0, bitwise equal at all 16 stations**	—
`item4` against `off`	$6.6 \cdot 10^{-11}$	station x2_0_x3_7.5, slip
`dense` against `off`	$2.7 \cdot 10^{-11}$	station x2_0_x3_7.5, V
`dense` against `item4`	$6.6 \cdot 10^{-11}$	station x2_0_x3_7.5, slip

Each difference is the largest $|a - b|$ over the largest $|a|$ of that quantity at that station. The three runs also print the same end state: 7394 steps to t = 1.00004 s, max $|V|$ = 3.7397 m/s, σ_n^{eff} in [108.368, 128.537] MPa. The `item4` difference is the one job 1078825 found for the same flags ($6.625 \cdot 10^{-11}$, at the same station and quantity), and the dense blocks add none larger. As in the cloud, the change is round-off and no new gold is needed.

6.3 Speed

Run	ms per step (slowest rank)	Speedup of `dense`	Fault work, busiest rank	Fault output per frame	Driver wall time (1 s)	Setup and the rest
`off`	27.28	2.91×	6.78 ms	0.419 s	322.1 s	100 s
`item4`	16.91	**1.81×**	3.02 ms	0.123 s	257.0 s	126 s
`dense`	**9.36**	—	1.27 ms	0.122 s	196.9 s	122 s

The step time is the time inside `ode.Step` after step 1, divided by the step count, on the rank where it is largest (the driver's timing line). "Setup and the rest" is the wall time minus the time in `ode.Step` and in the fault output (49 frames); it holds the setup (mesh reading, partitioning, assembly) and the rest of the run outside the steps, which this job does not break down. Job 1078825 measured 27.00 ms (`off`) and 16.38 ms (`on`, the same flags as `item4`), so repeat jobs differ by 1 to 3%.

Against the predictions. The design predicted 1.8 to 2.2 times per step (design §4); Vista gives 1.81 times, at the bottom of that range. The cloud gave 2.20 times for TPV205 at p = 2 (3000 m mesh, 4 ranks). The 16-rank bench gives 1.59 times for the products alone (§6.5); the step gains more than that on the full nodes.

A full run (inferred). Gold 1075916 is the same case to 12 s: 88,725 steps, about 577 fault frames, 44 min 54 s on the same 16 nodes. Scaling the measured step time, setup and output time per frame of each run to that length gives:

Run	Stepping	Fault output	Setup and the rest	Total	SU (16 nodes, 0.33 SU per node-hour)
`off`	2421 s	242 s	100 s	46 min	4.1
`item4`	1501 s	71 s	126 s	28 min	2.5
`dense`	831 s	70 s	122 s	**17 min**	**1.5**

The `off` estimate is within 3% of the gold job's measured 44 min 54 s, which ran that code. The others are estimates, not runs: they assume the step time stays the same during the rupture as in the first second.

Fault work. The busiest rank's fault work fell from 3.02 ms (`item4`) to 1.27 ms (`dense`), although item 8a does not change the fault code. In the cloud it did not move (1.30 and 1.28 ms, TPV205 p = 2). I have not found the cause. Two readings fit, neither checked: on a full node the fault work is slowed by the memory traffic of the products, which the blocks reduce; or the timed region includes waiting for other ranks, which finish their products sooner.

6.4 Memory

Run	First node, peak in use	Slurm MaxRSS of the run	Dense blocks
`off`	138.6 GiB of 237.6	109.9 GiB	—
`item4`	138.6 GiB	36.0 GiB	—
`dense`	138.2 GiB	30.3 GiB	68.9 GiB over all ranks: 4.3 GiB per node, 39.1 kB per tet

The first-node peak is sampled every 2 s from `/proc/meminfo` (the job prints it as "GB"; it is 2^{30} bytes). The blocks do not raise it (138.2 against 138.6 GiB), so the peak is set elsewhere in setup; job 1078825 recorded 147 and 146 GiB for the same two older runs, so this number moves by several GiB between jobs. Slurm's MaxRSS falls in the same order as the stepping memory in the cloud (§5.3), but I have not checked how Vista's accounting sums it over the 144 tasks of a node, so I draw no ratio from it. The blocks take 39.1 kB per tet here against 35 kB on the 4-rank cloud mesh, because at 822 tets per rank a larger share of the faces are shared faces, which are stored once on each side (inferred from the layout, §3.2).

6.5 The bench on 16 ranks (build 1079599)

TPV205 1000 m mesh, p = 1, 187,606 tets on 16 ranks of one node, 60 alternating pairs, minimum (median) times:

Product	Old	Dense blocks	Speedup
$Ku + K_\eta v$ against the two hypre products	13.87 (14.20) ms	9.146 (9.293) ms	1.52× (1.53×)
$Ku + K_\eta v$ against item 4's sweep	15.88 (16.42) ms	10.12 (10.31) ms	1.57× (1.59×)
Ku alone against hypre	6.565 (6.703) ms	5.375 (5.495) ms	1.22× (1.22×)

All 14 checks passed: the dense products equal the hypre ones to $6.7 \cdot 10^{-16}$, $5.4 \cdot 10^{-16}$ and $6.6 \cdot 10^{-16}$ of their largest entry, with 23,046 shared-face blocks on the 16 ranks; the largest asymmetry is $4.0 \cdot 10^{-17}$ (K) and $3.4 \cdot 10^{-17}$ (K_η); the blocks take 6.0 kB per tet against 17.2 kB for the two CSR matrices. On this node with 16 ranks, item 4's sweep is slower than the two hypre products (0.91 times; job 1078847 measured 1.02 times on the same bench, so this ratio moves between jobs), and the gains of the blocks are smaller than in the cloud (2.40 and 1.92 times at 4 ranks): the two hypre products stream 14.5 GB/s per rank, 70% of the bench's triad (20.6 GB/s per rank), while the blocks stream 7.7 GB/s, 37% of it. With 16 ranks the node's memory is not the limit, so the blocks' smaller size counts for less (inferred from bytes and times).

6.6 What limits the step now (inference)

Of the 9.36 ms step, 1.27 ms is fault work; the driver does not time the products separately, so the split of the other 8.09 ms between the products and the remaining vector work is not measured on Vista. At 822 tets per rank the blocks are 32 MB per rank. If the whole 8.09 ms were the products, they would stream 4.0 GB/s per rank (570 GB/s per node); the same bound gives 4.2 and 5.2 GB/s for the CSR products of `off` and `item4` (with the design's 104 and 87 kB per tet at p = 2). The three are within 30% of each other although the 16-rank bench shows the blocks streaming at about half the CSR rate when memory is not the limit. That fits a shared memory-bandwidth limit on the full nodes, which would also explain why the step gains more than the 16-rank bench. It is not proof: no triad was run with 144 ranks on a Vista node, and the products' share of the step is not measured. If the limit is bandwidth, the next gain on the products needs fewer bytes, which is what option (b) offers (§7).

7. Limits and next steps

- **Machines.** §4 and §5 are from the 4-core cloud VM; §6 is Vista at 2304 ranks, TPV205 at p = 2 only. TPV104, TPV102 and p = 1 have not run on Vista with the dense blocks.
- **Orders above 2.** p = 3 to 5 run the generic loop, which no test has run yet. Higher orders are refused (Init returns false) and use the old path.
- **Diagnostics.** As with item 4, a diagnostic that calls `ApplyStiffness` (the energy monitor) assembles K again and keeps it, which gives back the memory but not the speed.
- **Option (b), matrix-free.** Not started; it waits on your decision. (b) reads 13 to 100 times fewer bytes than (a2) but does more arithmetic, so it can only win where memory bandwidth limits (a2). The Vista numbers do not settle whether it does on a full node (§6.6): they fit a bandwidth limit, but the share of the 9.36 ms step taken by the products was not measured there. The design's phase P0 (a p = 1 prototype of (b), timed against (a2) on one full Vista node) is the measurement that decides it.
- **Item 7 (local time stepping)** is not started; it waits on your decision (design §5).
- **Threads and GPU** (design P6) are out of scope.

8. Where it lives

- This report: `/mnt/project-files/profiling/nitsche-dense-blocks-8a-rev1.1.pdf`, with `nitsche-dense-blocks-8a-rev1.1-source.html` beside it; revision 1 stays beside it. Proposed for the repository as `miniapps/seas/document/nitsche_profiling_dev/nitsche-dense-blocks-8a-rev1.1.pdf`.
- Code: branch `claude/nitsche-profiling-w336yq`, `241cc80` (the dense blocks, the operator and driver switch, the bench section and the Makefile line) and `7cab085` (the Vista job and its README rows), on `d30c76e`. Pushed on 18 Oct 2026 after Chunhui Zhao's approval ("approve item 8a and the Vista run"), byte-identical to the reviewed copy.
- Vista: clone `$WORK/seas-mfem-profiling` at `7cab085`, account EAR26024. Build job 1079599 and speed job 1079640 (submitted on Chunhui Zhao's "Submit the dense speed job"). The speed job's stdout is proposed for `jobs/vista_profiling/results/tpv205_200m_p2_dense_1079640.out`, as relayed from Vista; the bench log stays on Vista (`miniapps/seas/bench_np16_1079599.log` in the clone), and the relayed copies of both are in `/mnt/project-files/profiling/dense-8a/vista/`. The run directories stay in `$SCRATCH/nitsche_profiling/tpv205_200m_p2_dense_1079640/{off,item4,dense}`; scratch is purged, and these runs are not golds.
- Cloud runs: `/mnt/project-files/profiling/dense-8a/` holds the run summary (`matrix8_summary.json`), the matrix log, the bench logs and the gate-test summary. The run directories and the reference build of `d30c76e` stay in the session scratchpad (`k8/`).